

University of Pisa

Department of Computer Science
Bachelor's Degree

Bachelor's thesis

A FRI-Based Polynomial Commitment Scheme

Supervisors:

Prof. Anna Bernasconi
University of Pisa

Dott. Giorgio Dell'Immagine
zSkSecurity

Candidate:

Sofia Scalzo
University of Pisa

Academic year 2025/2026

Abstract

This thesis presents a polynomial commitment scheme based on the FRI protocol over the BabyBear field, requiring no trusted setup and relying only on a hash function, which makes it transparent and plausibly post-quantum. The construction is developed from Reed-Solomon codes and the interactive oracle proof model up to FRI, its soundness, and the quotient technique that turns a low-degree test into a proof of evaluation. Its soundness is analysed in the provable regime, following the recent round-by-round analysis of the protocol. The construction is implemented and evaluated, demonstrating its practical efficiency in terms of proof size, prover time, and verifier time.

Contents

1 Introduction **5**
- 1.1 Contribution . . . 6
- 1.2 Structure of the Thesis . . . 6

2 Mathematical and Cryptographic Background **7**
- 2.1 Finite Fields . . . 7
  - 2.1.1 The BabyBear Field . . . 7
  - 2.1.2 Roots of Unity . . . 7
  - 2.1.3 Field Extensions . . . 8
- 2.2 Polynomials . . . 8
  - 2.2.1 The Number Theoretic Transform . . . 8
  - 2.2.2 Low-Degree Extension . . . 9
  - 2.2.3 Vanishing Polynomials . . . 9
- 2.3 Coding Theory . . . 9
- 2.4 Reed-Solomon Codes . . . 10
  - 2.4.1 Relative Hamming Distance . . . 10
- 2.5 Cryptographic Hash Functions . . . 11
- 2.6 Proof Systems . . . 11
  - 2.6.1 Interactive Proofs . . . 11
  - 2.6.2 Interactive Arguments . . . 12
  - 2.6.3 The Fiat-Shamir Transformation . . . 12
- 2.7 Polynomial Commitment Schemes . . . 13

3 Interactive Oracle Proofs and the BCS Transformation **14**
- 3.1 Merkle Trees . . . 14
  - 3.1.1 Openings and the Authentication Path . . . 14
  - 3.1.2 A Vector Commitment . . . 15
- 3.2 Interactive Oracle Proofs . . . 16
- 3.3 The BCS Transformation . . . 17

4 The FRI Protocol **19**
- 4.1 Overview and Problem Statement . . . 19
- 4.2 The Evaluation Domain . . . 20
  - 4.2.1 Requirements . . . 20
  - 4.2.2 Construction . . . 20
- 4.3 The Folding Operation . . . 22
  - 4.3.1 Even-Odd Decomposition . . . 22

4.3.2 Recovering the Components from a Pair . . . 23
4.3.3 The Fold . . . 23
4.4 The Protocol . . . 24
4.4.1 Commit Phase . . . 24
4.4.2 Query Phase . . . 25
4.4.3 Verification and the Colinearity Check . . . 25
4.5 Soundness . . . 26
4.5.1 The Two Things That Can Go Wrong . . . 26
4.5.2 Why Folding Preserves Distance . . . 27
4.5.3 The Soundness Theorem . . . 27
4.5.4 Beyond the Provable Regime . . . 28

5 FRI as a Polynomial Commitment Scheme **29**
5.1 From Low-Degree Testing to Evaluation Proofs . . . 29
5.2 The Quotient Technique . . . 30
5.3 Commit, Open, and Verify . . . 31

6 Implementation **33**
6.1 Architecture . . . 33
6.2 Cryptographic Components . . . 34
6.2.1 Merkle Commitments . . . 34
6.2.2 The Duplex-Sponge Challenger . . . 34
6.3 Optimisations . . . 35
6.4 Parameters and Security Level . . . 36
6.5 The Polynomial Commitment Scheme . . . 37
6.6 Evaluation . . . 38
6.7 Testing . . . 39

7 Conclusions **40**

Bibliography **41**

Chapter 1

Introduction

When a computation is delegated to a party we do not fully trust, we would like a way to check that it was performed correctly without redoing the work ourselves. Proof systems make this possible: a prover produces a short certificate that convinces a verifier of the correctness of a statement, and the verifier checks it far more cheaply than recomputing the statement from scratch. Over the last decade these systems have moved from theory to practice, underpinning scalable and privacy-preserving applications, and much of their machinery rests on a single building block, the *polynomial commitment scheme*.

These systems are no longer only a theoretical construction, and the polynomial commitment scheme is the component on which a great many of them are built. The most visible application is the scaling of blockchains through *zk-rollups*: instead of having every node re-execute thousands of transactions, a single succinct proof attests that the whole batch was processed correctly, and the base chain only verifies that proof. StarkNet, a Layer-2 network on Ethereum, does exactly this using STARK proofs, which are based on the same techniques underlying the FRI protocol studied in this thesis; other systems such as Polygon zkEVM follow a similar approach. A second family of applications combines these proofs with additional techniques to reveal that a statement is true without disclosing the underlying data: this enables privacy-preserving payments, identity and credential systems that prove a fact (for instance that a person is over a certain age) without disclosing the underlying document, and privacy-preserving analysis of sensitive data such as medical records. A third use is *verifiable computation*, where an expensive calculation is delegated to an untrusted server that returns the result together with a proof of its correctness.

FRI in particular has found uses beyond proving computations. A recent example is *data availability sampling* [HASW24], where a light node of a blockchain, which does not store the block contents, wants to be sure that those contents are actually available in the network without downloading them in full. By probing only a few random positions of an encoded block, and checking them against a short commitment, the node becomes convinced that the whole block can be recovered. Hall-Andersen, Simkin, and Wagner show that FRI can be turned into exactly such a scheme, which illustrates how the same low-degree testing machinery studied in this thesis serves as a building block far beyond its original purpose. What all these applications share is the same core need: a way to commit to data and later prove statements about it, cheaply

and reliably.

A polynomial commitment scheme lets a prover commit to a polynomial with a short digest and later prove its value at any chosen point, without requiring the verifier to know the polynomial in full. Different schemes make different trade-offs. Some rely on a trusted setup or on elliptic-curve assumptions that are broken by quantum computers. The scheme studied in this thesis avoids both: it is built on FRI, the Fast Reed-Solomon Interactive Oracle Proof of Proximity, and relies only a hash function. It is therefore *transparent*, requiring no trusted setup, and plausibly *post-quantum*, since its security rests on hashing alone.

1.1 Contribution

This thesis studies and explains from the ground up, over the BabyBear prime field $\mathbb{F}_p$ with $p = 2^{31} - 2^{27} + 1 = 15 \cdot 2^{27} + 1 = 2013265921$, and implements every part of it from scratch in Python without external cryptographic libraries. This includes the field arithmetic and its quartic extension, the Number Theoretic Transform, the Merkle commitments, a Keccak-based duplex-sponge challenger, and the FRI prover and verifier, on top of which the polynomial commitment scheme is realised through the quotient technique. The complete source code is publicly available at [Sca26].

Writing every component from scratch is a deliberate choice. Production systems are written in compiled languages and layered with optimisations, batching, and low-level tricks that make them fast but hard to follow. The goal here is the opposite: to expose clearly how a transparent proof system is put together. A self-contained implementation makes each step inspectable and keeps the distance between the mathematics and the code as small as possible, at the natural cost of the raw performance a production implementation would reach.

1.2 Structure of the Thesis

Chapter 2 sets up the mathematical and cryptographic background: finite fields, polynomials and the Number Theoretic Transform, coding theory and Reed-Solomon codes, hash functions, proof systems, and polynomial commitment schemes. Chapter 3 introduces the interactive oracle proof model and the BCS transformation that compiles it into a non-interactive argument using Merkle commitments. Chapter 4 presents the FRI protocol, its folding operation, and its soundness. Chapter 5 turns FRI into a polynomial commitment scheme through the quotient technique. Chapter 6 describes the implementation, its cryptographic components, its optimisations, and the parameters and security level. Chapter 7 draws some concluding remarks.

Chapter 2

Mathematical and Cryptographic Background

This chapter introduces the notation and mathematical foundations used throughout the thesis.

2.1 Finite Fields

Definition 2.1 (Prime Field)**.** Let p be a prime number. The *prime field* $\mathbb{F}_p$ is the set $\{0, 1, \ldots, p-1\}$ equipped with addition and multiplication modulo p.

The multiplicative group $\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$ is cyclic of order $p-1$. An element $g \in \mathbb{F}_p^*$ is called a *generator* if $\{g^0, g^1, \ldots, g^{p-2}\} = \mathbb{F}_p^*$. For any $a \in \mathbb{F}_p^*$, the multiplicative inverse can be computed via Fermat's little theorem:

$$a^{-1} \equiv a^{p-2} \pmod{p}. \tag{2.1}$$

Example 2.1. In $\mathbb{F}_7 = \{0, 1, \ldots, 6\}$ arithmetic wraps around modulo 7: for instance $5 + 4 = 2$ and $5 \cdot 4 = 6$. The inverse of 3 is $3^{p-2} = 3^5 = 5$, since $3 \cdot 5 = 15 = 1 \bmod 7$.

2.1.1 The BabyBear Field

The implementation described in this thesis operates over the BabyBear prime field, defined by

$$p = 2^{31} - 2^{27} + 1 = 2\,013\,265\,921. \tag{2.2}$$

Since $p < 2^{31}$, every field element fits in a 32-bit unsigned integer, and the product of two elements fits in a 64-bit integer.

The multiplicative group $\mathbb{F}_p^*$ has order $p-1 = 2^{27} \cdot 15$. The factor 2^{27}, the *two-adicity* of the field, means that $\mathbb{F}_p^*$ contains a cyclic subgroup of order 2^{27}, which is essential for the Number Theoretic Transform.

2.1.2 Roots of Unity

Definition 2.2 (Primitive Root of Unity)**.** An element $\omega \in \mathbb{F}_p$ is a *primitive N-th root of unity* if $\omega^N = 1$ and $\omega^k \neq 1$ for all $0 < k < N$.

Example 2.2. In $\mathbb{F}_7$ the element $\omega = 2$ is a primitive 3rd root of unity: $2^1 = 2$, $2^2 = 4$, $2^3 = 1$, and no smaller power gives 1. Its powers form the subgroup $\{1, 2, 4\}$ of order 3.

Given a primitive N-th root of unity ω, the set $H = \{1, \omega, \omega^2, \ldots, \omega^{N-1}\}$ is a multiplicative subgroup of $\mathbb{F}_p^*$ of order N. Because BabyBear has two-adicity 27, subgroups of order 2^k for any $k \leq 27$ exist and are generated from a fixed multiplicative generator g by taking $g^{(p-1)/2^k}$.

These subgroups serve as evaluation domains for the polynomials in the Scalable Transparent Argument of Knowledge (STARK) protocol: the execution trace is evaluated on a subgroup H of order 2^h, and the Fast Reed-Solomon Interactive Oracle Proof of Proximity (FRI) operates on a larger domain.

2.1.3 Field Extensions

Definition 2.3 (Field Extension)**.** A *field extension* $\mathbb{F}_{p^k}$ of degree k over $\mathbb{F}_p$ is the quotient ring $\mathbb{F}_p[X]/(f(X))$, where $f(X) \in \mathbb{F}_p[X]$ is an irreducible polynomial of degree k. Elements are polynomials of degree less than k with coefficients in $\mathbb{F}_p$.

In this work we use the *quartic extension* $\mathbb{F}_{p^4}$, constructed via the irreducible polynomial $X^4 - 11$. An element of $\mathbb{F}_{p^4}$ is a tuple $(c_0, c_1, c_2, c_3) \in \mathbb{F}_p^4$ representing $c_0 + c_1 X + c_2 X^2 + c_3 X^3$, with the reduction rule $X^4 = 11$. The extension has $p^4 \approx 2^{124}$ elements, large enough that a folding challenge sampled from it makes the soundness error negligible, as discussed in Chapter 4.

Example 2.3. A smaller instance is $\mathbb{F}_{49} = \mathbb{F}_7[X]/(X^2 + 1)$, which is a field because $X^2 + 1$ has no root in $\mathbb{F}_7$. Its elements are $a + bX$ with $a, b \in \mathbb{F}_7$ and the rule $X^2 = -1$. For example, $(2 + 3X)(1 + X) = 2 + 5X + 3X^2 = 2 + 5X - 3 = 6 + 5X$.

2.2 Polynomials

A univariate polynomial over $\mathbb{F}_p$ of degree at most d is $f(X) = \sum_{i=0}^{d} c_i X^i$ with $c_i \in \mathbb{F}_p$. It can be represented by its *coefficient vector* $(c_0, \ldots, c_d)$ or by its *evaluation vector* on a domain D. Converting between the two representations, and evaluating a polynomial at a point, are the fundamental operations of the prover.

2.2.1 The Number Theoretic Transform

The conversion between coefficient and evaluation representations is performed using the Number Theoretic Transform (NTT), the finite-field analogue of the Fast Fourier Transform. When the evaluation domain is a multiplicative subgroup $D = \{1, \omega, \ldots, \omega^{N-1}\}$ with N a power of 2, the NTT computes all evaluations $f(\omega^i)$ in $O(N \log N)$ operations. The algorithm exploits the decomposition

$$f(X) = f_{\text{even}}(X^2) + X \cdot f_{\text{odd}}(X^2),$$

recursing on the even- and odd-indexed coefficients over the squared subgroup, which has half the size. The inverse transform runs the same routine with ω^{-1} and divides by N.

2.2.2 Low-Degree Extension

The *low-degree extension* (LDE) of a polynomial is its evaluation on a domain larger than its coefficient count. Given the n coefficients of f, the LDE with blowup factor b evaluates f on a coset of a subgroup of size $b \cdot n$. The resulting evaluation vector is the Reed-Solomon codeword that the prover commits to. In our implementation the low-degree extension is computed by padding the coefficients with zeros, applying a coset shift, and running the NTT on the enlarged domain [Sta21, Section 3.4].

2.2.3 Vanishing Polynomials

Definition 2.4 (Vanishing Polynomial [Sta21, Section 5.1])**.** Given a set $S \subseteq \mathbb{F}$, the *vanishing polynomial* of S is $Z_S(X) := \prod_{s \in S}(X - s)$. When S is a multiplicative subgroup $H = \{1, \omega, \ldots, \omega^{N-1}\}$, this simplifies to $Z_H(X) = X^N - 1$.

Example 2.4. Take the subgroup $H = \{1, 2, 4\}$ of $\mathbb{F}_7^*$ from Example 2.2. Its vanishing polynomial is $Z_H(X) = (X-1)(X-2)(X-4) = X^3 - 1$, which evaluates to 0 at each of $1, 2, 4$ and is nonzero elsewhere, for instance $Z_H(3) = 5$.

Z_H vanishes on every element of H and nowhere else. If a constraint polynomial $c(X)$ must be zero on H, then $Z_H(X)$ divides $c(X)$, and the quotient $q(X) = c(X)/Z_H(X)$ is a polynomial. The verifier checks that q has low degree, which certifies that c vanishes on H [Sta21]. This mechanism is the basis of the STARK construction.

2.3 Coding Theory

We recall the standard notions from coding theory that the soundness analysis of the FRI protocol relies on. The specific family of codes used in this work, the Reed-Solomon codes, is introduced in Section 2.4 as an instance of these general definitions.

Definition 2.5 (Error-Correcting Code [Arn+24, Definition 3.1])**.** An *error-correcting code* of length n over an alphabet Σ is a subset $C \subseteq \Sigma^n$. The code is a *linear code* if $\Sigma = \mathbb{F}$ is a field and C is a linear subspace of $\mathbb{F}^n$. The *rate* of a linear code is $\rho := \dim(C)/n$.

Example 2.5. The set of all vectors $(a, b, a + b) \in \mathbb{F}_7^3$ is a linear code of length $n = 3$: it is a subspace of dimension 2 (the free choices a and b), so its rate is $\rho = 2/3$. The third coordinate is a built-in check that lets some errors be detected.

The elements of C are its *codewords*, and the *minimum distance* of a code is the smallest relative Hamming distance (Definition 2.8) between two distinct codewords. When a word lies far from every codeword, several codewords may still lie within a given distance of it; the following notion quantifies this.

Definition 2.6 (List Decoding [Arn+24, Definition 3.2])**.** For a code $C \subseteq \Sigma^n$, a word $f \in \Sigma^n$, and a proximity parameter $\delta \in [0, 1]$, the *list* of C at f within radius δ is

$$\Lambda(C, f, \delta) := \{u \in C : \Delta(f, u) \leq \delta\}.$$

The code is (δ, ℓ)-*list decodable* if $|\Lambda(C, f, \delta)| \leq \ell$ for every $f \in \Sigma^n$. A radius δ is *within unique decoding* if the code is $(\delta, 1)$-list decodable.

Example 2.6. Within the unique decoding radius there is at most one codeword near a received word, so the list $\Lambda(C, f, \delta)$ has size at most 1 and decoding is unambiguous. Past that radius the list can grow: a word sitting exactly between two codewords has both of them in its list, so the code is no longer $(\delta, 1)$-list decodable there.

For a code of minimum distance δ_C, any radius $\delta \leq \delta_C/2$ is within unique decoding. Beyond it, the list size is controlled, for the codes we use, by the Johnson bound stated below.

Theorem 2.1 (Johnson Bound [Arn+24, Theorem 4.3])**.** *The Reed-Solomon code of rate ρ is $\left(1 - \sqrt{\rho} - \eta,\ \frac{1}{2\eta\sqrt{\rho}}\right)$-list decodable for every $\eta \in (0, 1 - \sqrt{\rho})$.*

Example 2.7. For rate $\rho = 1/2$, the unique decoding radius is $\frac{1-\rho}{2} = 0.25$, while the Johnson radius is $1 - \sqrt{1/2} \approx 0.29$. Between these two values a received word may have a few codewords in its list, but the Johnson bound guarantees the number stays small, which is what the soundness analysis needs.

The two thresholds, the *unique decoding* radius $\frac{1-\rho}{2}$ and the *Johnson* radius $1-\sqrt{\rho}$, are exactly where the soundness guarantees of FRI change form (Chapter 4).

2.4 Reed-Solomon Codes

Definition 2.7 (Reed-Solomon Code [BS+18, Section 2.1])**.** For a finite field $\mathbb{F}$, a subset $S \subseteq \mathbb{F}$, and a rate parameter $\rho \in (0, 1)$, the *Reed-Solomon code* $\mathrm{RS}[\mathbb{F}, S, \rho]$ is the set of functions $f : S \to \mathbb{F}$ that are evaluations of polynomials of degree less than $\rho|S|$:

$$\mathrm{RS}[\mathbb{F}, S, \rho] = \{(f(x))_{x \in S} : f \in \mathbb{F}[X],\ \deg(f) < \rho|S|\}.$$

Example 2.8. Take $\mathbb{F}_7$, evaluation set $S = \{1, 2, 3, 4, 5, 6\}$, and rate $\rho = 1/3$, so codewords are evaluations of polynomials of degree less than $\rho|S| = 2$, that is, lines $a + bX$. The polynomial $f(X) = 2 + 3X$ gives the codeword $(f(1), \ldots, f(6)) = (5, 1, 4, 0, 3, 6)$. Any two distinct lines agree in at most one point, so their codewords differ in at least five of the six positions.

Each codeword is the evaluation of a polynomial of degree less than $\rho|S|$ on all points of S. The code has minimum distance $|S| - \rho|S| + 1$, so two distinct codewords differ on a large fraction of positions.

The parameter ρ is the *rate*, and $1/\rho$ is the *blowup factor*. In our implementation the blowup factor is 2 ($\rho = 1/2$): the codeword produced by the low-degree extension is twice the length of the coefficient vector.

2.4.1 Relative Hamming Distance

Definition 2.8 (Relative Hamming Distance)**.** The *relative Hamming distance* between two vectors $u, v \in \mathbb{F}^n$ is

$$\Delta(u, v) = \frac{|\{i : u_i \neq v_i\}|}{n}.$$

The *distance* of a vector u from a code C is $\Delta(u, C) = \min_{v \in C} \Delta(u, v)$. A vector u is δ-*close* to C if $\Delta(u, C) \leq \delta$, and δ-*far* if $\Delta(u, C) > \delta$.

Example 2.9. The codeword $u = (5, 1, 4, 0, 3, 6)$ of Example 2.8 and the corrupted word $v = (5, 1, 4, 2, 3, 6)$ differ in exactly one of six positions, so $\Delta(u, v) = 1/6$. If every genuine codeword differs from v in at least this many positions, then v is $\frac{1}{6}$-far from the code.

The FRI protocol tests proximity to a Reed-Solomon codeword. As stated in [BS+18, Section 2.1], the verifier's task is to distinguish between the case that $f \in \mathrm{RS}[\mathbb{F}, S, \rho]$ and the case that f is far from $\mathrm{RS}[\mathbb{F}, S, \rho]$ in relative Hamming distance.

2.5 Cryptographic Hash Functions

Definition 2.9 (Collision-Resistant Hash Function)**.** A *collision-resistant hash function* is a function $H : \{0,1\}^* \to \{0,1\}^\lambda$ such that no efficient algorithm can find distinct inputs $x \neq x'$ with $H(x) = H(x')$, except with negligible probability.

Example 2.10. SHA-256 maps any input to a 256-bit digest. Changing even one bit of the input produces an essentially unrelated digest, and no method is known to find two inputs with the same digest faster than the roughly 2^{128} tries of a birthday attack, which is what collision resistance requires.

Hash functions serve two roles in STARK systems: constructing Merkle trees for committing to polynomial evaluations, and deriving the verifier's challenges via the Fiat-Shamir heuristic. Our implementation uses two different hash functions for these two roles: SHA-256 for the Merkle commitments and Keccak-f[1600] for the challenger.

The distinction matters for the Fiat-Shamir transformation. Chiesa and Orrù [CO25] prove that a duplex sponge construction (such as Keccak/SHA-3) efficiently supports the Fiat–Shamir transformation [CO25], whereas Merkle-Damgård constructions (such as SHA-256) require additional overhead. For this reason we use a Keccak-based sponge for the challenger, while retaining SHA-256 for the Merkle trees, where only collision resistance is required.

2.6 Proof Systems

We now establish the formal framework of proof systems, following Thaler [Tha22].

2.6.1 Interactive Proofs

Definition 2.10 (Interactive Proof [Tha22, Definition 3.1])**.** An *interactive proof system* for a language $\mathcal{L}$ is a pair $(\mathcal{P}, \mathcal{V})$ where $\mathcal{P}$ is a computationally unbounded prover and $\mathcal{V}$ is a probabilistic polynomial-time verifier. The protocol proceeds in rounds, and must satisfy:

- **Completeness.** For every $x \in \mathcal{L}$, $\Pr[\mathcal{V} \text{ accepts}] \geq 1 - \delta_c$.
- **Soundness.** For every $x \notin \mathcal{L}$ and every cheating prover $\mathcal{P}^*$, $\Pr[\mathcal{V} \text{ accepts}] \leq \delta_s$.

Example 2.11. A classic interactive proof is for graph non-isomorphism. The verifier secretly picks one of two graphs, randomly relabels its vertices, and sends the result to the prover, who must say which graph it came from. If the graphs are non-isomorphic an unbounded prover always answers correctly; if they are isomorphic no prover can do better than guessing, so the verifier catches a false claim with probability $1/2$ per round.

2.6.2 Interactive Arguments

Relaxing the soundness of an interactive proof to hold only against computationally efficient provers yields a weaker but more flexible notion.

Definition 2.11 (Interactive Argument [Tha22, Section 3.2])**.** An *interactive argument* for a language $\mathcal{L}$ is an interactive proof in which the soundness condition is only required to hold against prover strategies that run in probabilistic polynomial time.

Example 2.12. The FRI-based systems in this thesis are arguments, not proofs. Their soundness rests on the collision resistance of a hash function: a prover with unlimited time could find hash collisions and forge a convincing transcript, but a realistic polynomial-time prover cannot, so soundness holds against every efficient prover.

Both models share the same structure. In a *public-coin* protocol, every verifier message is a uniformly random challenge, independent of the prover's messages, and the verifier's decision is a deterministic function of the *transcript*, the ordered sequence of messages and challenges. These are exactly the protocols to which the Fiat-Shamir transformation (Subsection 2.6.3) applies. An *argument of knowledge* further requires an efficient extractor E that recovers a valid witness from any convincing prover, ensuring that the verifier is convinced that the prover indeed knows the witness.

STARKs are *arguments*, not *proofs* in the formal sense: their soundness relies on the collision resistance of a hash function, which a computationally unbounded prover could break. The interactive oracle proofs of Section 3.2 specialise this model by providing the prover's messages as oracles.

2.6.3 The Fiat-Shamir Transformation

A *non-interactive* proof consists of a single message from the prover to the verifier. The Fiat-Shamir transformation removes interaction from a public-coin protocol by replacing the verifier's random challenges with the output of a hash function applied to the transcript so far. Under the random oracle model, this preserves soundness. Chiesa and Orrù [CO25] give a formal treatment using a duplex sponge, which efficiently handles the transcript using the sponge's native *absorb* and *squeeze* operations. Our challenger implements this construction over Keccak-$f[1600]$.

which efficiently handles the transcript using the sponge's native absorb and squeeze operations

2.7 Polynomial Commitment Schemes

Definition 2.12 (Polynomial Commitment Scheme)**.** A *Polynomial Commitment Scheme* (PCS) consists of four algorithms:

- $\mathsf{Setup}(\lambda, d) \to \mathsf{pp}$: generates public parameters for polynomials of degree at most d.
- $\mathsf{Commit}(\mathsf{pp}, f) \to C$: produces a commitment C to the polynomial f.
- $\mathsf{Open}(\mathsf{pp}, f, z) \to (v, \pi)$: produces a proof π that $f(z) = v$.
- $\mathsf{Verify}(\mathsf{pp}, C, z, v, \pi) \to \{0, 1\}$: checks the evaluation proof.

The scheme must satisfy correctness and binding: correctness ensures that an honestly generated proof for a true evaluation always verifies, while binding guarantees that no efficient prover can open a commitment to two different values at the same point.

Example 2.13. Let $f(X) = 5 + 2X + 3X^2$ over $\mathbb{F}_{17}$. The prover first commits to f, producing a commitment C that fixes the polynomial without revealing its coefficients. Later the verifier names a point, say $z = 4$, and asks for the value of f there. The prover replies with $v = f(4) = 5 + 2 \cdot 4 + 3 \cdot 16 = 61 = 10 \pmod{17}$, together with a proof π.

The proof π convinces the verifier that $v = 10$ is the value at $z = 4$ of the exact polynomial committed in C, and not of some other polynomial the prover might substitute; the verifier becomes convinced of the single equality $f(4) = 10$ without ever seeing the coefficients of f. The binding property is what makes this meaningful: the prover cannot open the same commitment C to two different values at the same point, so it could not also convince the verifier that $f(4) = 11$. How π is actually built and checked, using the FRI protocol, is the subject of Chapter 5.

The STARK system implemented in this thesis uses a *FRI-based PCS* [BS+18]: a polynomial is committed via a Merkle tree over its low-degree extension, and evaluation proofs are constructed through the FRI low-degree testing protocol. This approach requires no trusted setup (transparent) and relies only on hash functions (post-quantum secure). The FRI protocol is described in detail in Chapter 4.

Chapter 3

Interactive Oracle Proofs and the BCS Transformation

The FRI protocol of Chapter 4 is an *interactive oracle proof* (IOP): the prover sends oracles that the verifier queries at a few positions, rather than messages it reads in full. This chapter introduces the IOP model and the BCS transformation that compiles an IOP into a non-interactive argument by realising its oracles as Merkle commitments and its queries as commitment openings. Once this compiler is in place, the FRI protocol can be described purely in terms of oracles and queries, leaving the commitment machinery implicit.

3.1 Merkle Trees

A Merkle tree commits to a sequence of values by hashing them into a tree. The whole sequence is represented by one value, the root, and any position can later be revealed with a short proof. This is the device that realises the oracles of an IOP.

Definition 3.1 (Merkle Tree)**.** A *Merkle tree* over a sequence $(v_0, v_1, \ldots, v_{N-1})$ is a complete binary tree where each leaf stores the hash of one element, $\ell_i = H(v_i)$; each internal node stores the hash of the concatenation of its two children, $H(n_{\text{left}} \| n_{\text{right}})$; and the root r is a single digest that binds the entire sequence.

The root has fixed length and depends on every element of the sequence. Changing any v_i changes its leaf hash $H(v_i)$, which changes every node on the path from that leaf to the root, and so changes the root. The root therefore *binds* the sequence: this is made precise as position binding in Theorem 3.1 below, once openings have been defined.

3.1.1 Openings and the Authentication Path

To reveal the value at a position without sending the whole sequence, the prover uses an authentication path. To open position i, it reveals v_i and the *authentication path*: the sibling hashes along the route from the leaf ℓ_i to the root, one per level, so $O(\log N)$ hashes in total.

With these siblings the verifier can recompute the root on its own, even though it never sees the other leaves. It starts from $\ell_i = H(v_i)$ and, at each level, hashes the value it currently holds together with the provided sibling, in the correct left-right order, obtaining the parent node; repeating this up the tree yields a candidate root, which it compares with the committed one. A single sibling per level suffices because each node on the path is the hash of two inputs, the node and its sibling; the sibling is the one input the verifier does not already hold. The verifier accepts if the recomputed root equals the committed root. The cost is $O(\log N)$ hash evaluations.

We formalise this recomputation in Algorithm 1. The order in which each sibling is combined, whether the current node goes on the left or on the right, is dictated by the bits of the position i: at level ℓ, the ℓ-th bit of i (counting from the leaf) is 0 when the current node is a left child, so the sibling is hashed on its right, and 1 when it is a right child, so the sibling goes on its left.

Algorithm 1 RootFromPath: recompute the root from an opening.

Require: value v_i, position i, sibling hashes $(s_1, \ldots, s_d)$

$node \leftarrow H(v_i)$ ▷ start from the leaf
for $\ell = 1$ **to** d **do**
  if $\mathrm{bit}_\ell(i) = 0$ **then**
    $node \leftarrow H(node \,\|\, s_\ell)$ ▷ current node is a left child
  else
    $node \leftarrow H(s_\ell \,\|\, node)$ ▷ current node is a right child
  end if
end for
return *node* ▷ accept iff equal to the committed root

Example 3.1. For four values (v_0, v_1, v_2, v_3) the tree has leaves $\ell_i = H(v_i)$, two internal nodes $H(\ell_0\|\ell_1)$ and $H(\ell_2\|\ell_3)$, and a root hashing the two. To open position 2, the prover reveals v_2 and the two siblings $s_1 = \ell_3$ and $s_2 = H(\ell_0\|\ell_1)$; here the depth is $d = 2$ and the position in binary is $i = 2 = (10)_2$.

Running algorithm 1 on this input:

- start: $node = H(v_2) = \ell_2$;
- level 1: $\mathrm{bit}_1(2) = 0$, so ℓ_2 is a left child and $node \leftarrow H(\ell_2\|s_1) = H(\ell_2\|\ell_3)$;
- level 2: $\mathrm{bit}_2(2) = 1$, so the current node is a right child and $node \leftarrow H(s_2\|node) = H\big(H(\ell_0\|\ell_1)\|H(\ell_2\|\ell_3)\big)$.

The final value is the root, which the verifier compares with the committed one (Figure 3.1). The two revealed siblings were exactly what was missing at each level to move one step up.

3.1.2 A Vector Commitment

A Merkle tree is a *vector commitment*. The prover commits to a vector with the root, and later opens any position with an authentication path. Its security property is

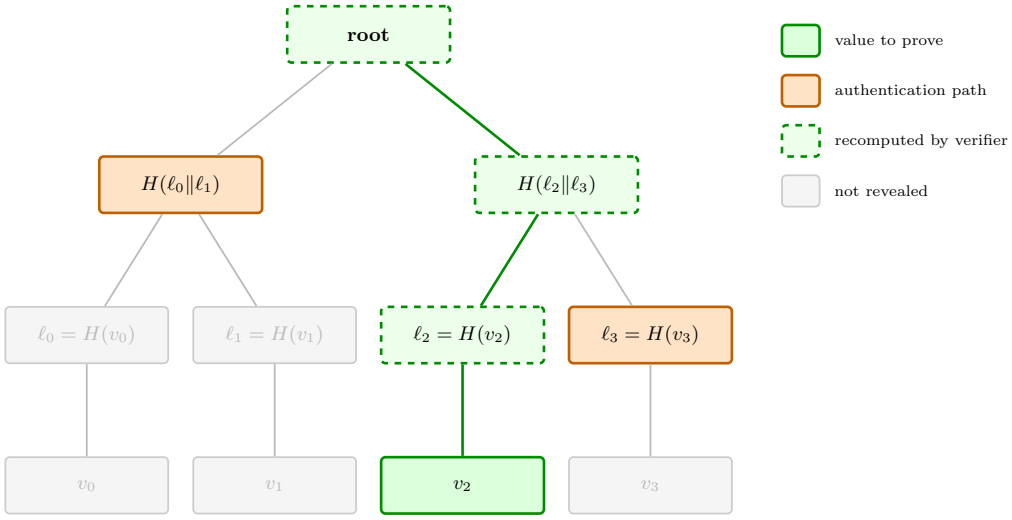


Figure 3.1: Verification of v_2 in a four-leaf Merkle tree. The prover reveals the *authentication path* (the two siblings ℓ_3 and $H(\ell_0\|\ell_1)$, in orange); the verifier recomputes the dashed nodes from v_2 up to the root and compares the result with the committed root. The greyed-out nodes are never revealed.

position binding: after publishing a root, the prover cannot open one position to two different values.

Proposition 3.1 (Position Binding)**.** *Let H be a random oracle. No efficient prover can output a root r, a position i, a length N, and two openings (v, p) and (v', p') for position i with $v \neq v'$ such that both recompute to r, except with negligible probability.*

By position binding, a Merkle root can stand in for an oracle. The committed vector is fixed, and the prover cannot change the value at a queried position after committing.

3.2 Interactive Oracle Proofs

An interactive oracle proof is an interactive proof, as defined in Subsection 2.6.1, in which the verifier has oracle access to the prover's messages rather than reading them in full. As there, prover and verifier exchange messages over several rounds; the new element is that the verifier does not read the prover's messages in full, but is given *oracle access* to them, querying only a few chosen positions.

Definition 3.2 (Interactive Oracle Proof [BSCS16, Definition 2.2])**.** An *Interactive Oracle Proof* (IOP) for a language $\mathcal{L}$ is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ that proceeds in rounds. In each round:

1. the prover sends an *oracle message* π_i (a function that the verifier may query at chosen positions);

2. the verifier sends a random challenge r_i.

After k rounds, the verifier makes a bounded number of queries to $\pi_1, \ldots, \pi_k$ and outputs accept or reject.

Example 3.2. FRI is itself an IOP. In each round the prover's oracle is a codeword, and the verifier's challenge is a folding coefficient; at the end the verifier queries only a handful of positions of each oracle rather than reading the full codewords, which is what makes verification succinct. The oracles are treated abstractly here; their concrete realisation as Merkle commitments is deferred to the BCS transformation.

The key difference from a standard interactive proof is that the verifier has *oracle access* to the prover's messages: rather than receiving each message in full, it may query a small number of chosen positions of π_i. The oracle fixes the message in advance, ensuring consistent answers to every query, but leaves the realisation unspecified. The BCS transformation of Section 3.3 instantiates these oracles concretely; until then, we reason about FRI in terms of oracles and queries alone.

An IOP of *proximity* (IOPP) is an IOP in which the verifier tests whether the prover's initial oracle is close to a codeword of a specified code, rather than testing membership in a language [BS+18]. FRI, described in Chapter 4, is an IOPP for Reed-Solomon codes.

3.3 The BCS Transformation

The IOP model of Section 3.2 leaves two things abstract: the oracle messages, which the verifier can query but not read in full, and the interaction, which requires the verifier to send live random challenges. The BCS transformation [BSCS16] removes both, compiling an IOP into a non-interactive argument in the random oracle model. It does so by realising each abstract ingredient with a concrete one.

- **Oracles become commitments.** Each oracle message π_i is replaced by a Merkle commitment to its values (Section 3.1): the prover builds a Merkle tree over π_i and sends only the root. By position binding, the root fixes the message so that it cannot be changed after the fact, which is the role the oracle played in the IOP. This is where the Merkle trees commit to the oracle messages: in FRI the prover evaluates a polynomial on the FRI domain, builds a Merkle tree over the evaluations, and sends the root to the verifier.

- **Queries become openings.** When the verifier queries position j of π_i, the prover answers with the value together with its authentication path, and the verifier checks the path against the committed root. An opening that verifies is a consistent answer to an oracle query.

- **Challenges become random-oracle outputs.** Using the Fiat–Shamir transformation (Subsection 2.6.3), each verifier challenge is set to the output of the random oracle on the transcript so far, so no live interaction is needed and the whole exchange becomes a single message. The challenge of a round is fixed only after the commitment of that round is in the transcript, which preserves soundness.

Carried out on a round-by-round sound IOP, the two steps give the sequence

$$\text{IOP} \xrightarrow{\text{BCS}} \text{interactive argument} \xrightarrow{\text{Fiat-Shamir}} \text{non-interactive argument},$$

all in the random oracle model, where the soundness error of the final argument is that of the IOP together with the binding error of the Merkle commitments.

Because of this compiler, once an IOP is specified we may describe it at the level of oracles and queries: it is enough to say that the prover sends the oracle for a claimed codeword and that the verifier queries it at chosen positions, with the understanding that the oracles are Merkle commitments and the queries their openings. We adopt this style in Chapter 4, where the resulting FRI IOP is compiled into a non-interactive argument through the transformation described here.

Chapter 4

The FRI Protocol

This chapter presents the Fast Reed-Solomon Interactive Oracle Proof of Proximity, the protocol at the core of the system implemented in this thesis. We begin by stating the problem FRI solves, then describe the algebraic structure of the evaluation domain, and finally the folding operation that drives the protocol.

4.1 Overview and Problem Statement

Recall from Definition 2.7 that, for a subset $S \subseteq \mathbb{F}$ and a rate parameter $\rho \in (0, 1)$, the Reed-Solomon code $\mathrm{RS}[\mathbb{F}, S, \rho]$ consists of the evaluations on S of polynomials of degree less than $\rho|S|$. The problem FRI addresses is the following.

Definition 4.1 (Rate Proximity Testing (RPT) [BS+18, Section 2.1])**.** A verifier is given oracle access to a function $f : S \to \mathbb{F}$, together with auxiliary information supplied by a prover. The verifier must distinguish, with high probability and using only a small number of queries, between the case that $f \in \mathrm{RS}[\mathbb{F}, S, \rho]$ and the case that f is far from every member of $\mathrm{RS}[\mathbb{F}, S, \rho]$ in relative Hamming distance (Definition 2.8).

RPT is a special case of the low-degree testing problem, and Ben-Sasson et al. identify it as a major bottleneck for transparent proof systems [BS+18, Section 2.1].

A naive solution is instructive. The verifier could ask the prover to open f at $\rho|S|$ positions, interpolate the unique polynomial of degree less than $\rho|S|$ through them, and check that it agrees with f elsewhere. This fails on two counts: the number of openings is proportional to the degree bound, which can reach 2^{20} or more in practical instances, and every opening carries a Merkle authentication path of $\log |S|$ hash values (Section 3.1), so the proof size becomes prohibitive.

FRI takes a different route. Instead of testing the degree directly, it *reduces* it: the prover repeatedly halves the degree of the committed polynomial using randomness supplied by the verifier, until the degree is small enough to send the remaining polynomial directly. Each reduction step is committed, and the verifier spot-checks the consistency of the steps at a small number of random positions. If the original function is far from the code, then with high probability at least one reduction is inconsistent, and the verifier detects it.

This yields a protocol with prover arithmetic complexity strictly linear in $|S|$ and verifier arithmetic complexity strictly logarithmic [BS+18, Section 2.1]. In the ter-

minology of Section 3.2, FRI is an interactive oracle proof of proximity (IOPP) for Reed-Solomon codes: the prover's messages are committed codewords which the verifier queries at a bounded number of positions, rather than reading in full.

Remark 4.1. FRI establishes *proximity*, not membership. A successful verification does not certify that the committed vector is exactly a Reed-Solomon codeword, only that it is close to one in relative Hamming distance. This matches the promise problem RPT is stated as: the verifier must separate words that are codewords from words that are far from every codeword, and nothing is required in the intermediate range. Proximity is the strongest guarantee attainable, because a codeword and the same codeword altered in a single position differ in only one of N places, and a verifier reading a sublinear number of positions almost never queries that place. It is also sufficient for the intended use: within the unique decoding radius, the radius $\frac{1-\rho}{2}$ below which at most one codeword lies near any word, a vector close to the code determines a unique low-degree polynomial, which the polynomial commitment scheme of Chapter 5 uses in its place. This distinction is central to the soundness analysis of Section 4.5.

4.2 The Evaluation Domain

The folding step of FRI relies on specific algebraic structure in the evaluation domain. We state the requirements first, then give the construction used in our implementation.

4.2.1 Requirements

The reduction from a polynomial f to a polynomial of half the degree is carried out by pairing the evaluations of f at x and at $-x$. For this to be possible at every position, the domain must be closed under negation. Moreover, the folded polynomial is committed not on D but on the squared domain $D^{(2)} := \{x^2 : x \in D\}$, and since the folding step is applied iteratively, $D^{(2)}$ must be closed under negation as well, and so on for every subsequent layer [Sta21, Section 3.9.1].

We therefore require a chain of domains

$$D = D_0, \quad D_{i+1} = D_i^{(2)}, \quad i = 0, 1, \ldots$$

such that every D_i is closed under negation and $|D_{i+1}| = |D_i|/2$. These requirements are met by taking D to be a coset of a multiplicative subgroup of order 2^k [Sta21, Section 3.9.1].

A coset is used rather than the subgroup itself so that the evaluation domain is disjoint from the trace domain on which the polynomial being committed is defined [Sta21, Section 3.4]. Disjointness is required in the full STARK protocol, where constraint polynomials are expressed as rational functions whose denominators vanish on the trace domain; evaluating them on a disjoint domain keeps those denominators nonzero.

4.2.2 Construction

We build the domain from the roots of unity of Subsection 2.1.2. Let n be the number of coefficients of the polynomial to be committed, b the blowup factor, and $N = b \cdot n = 2^k$

the size of the evaluation domain. Two elements are fixed:

$$\omega = \gamma^{(p-1)/2^k}, \qquad g = \gamma^{(p-1)/2^{k+1}}, \tag{4.1}$$

where γ is a fixed multiplicative generator of $\mathbb{F}_p^*$. Here ω is a primitive 2^k-th root of unity: its powers $\langle\omega\rangle = \{1, \omega, \ldots, \omega^{N-1}\}$ form a cyclic group of N equally spaced points. The element g is a primitive 2^{k+1}-th root of unity, one "half-step" finer, and it plays the role of an *offset* that rotates the whole group. The two are linked by the invariant

$$g^2 = \omega, \tag{4.2}$$

which, as we show below, is exactly what makes the domain reproduce itself under squaring.

The evaluation domain is the coset

$$D = g \cdot \langle\omega\rangle = \{\, g\omega^j : j = 0, \ldots, N-1 \,\}, \tag{4.3}$$

i.e. the group $\langle\omega\rangle$ shifted by the offset g. The codeword committed by the prover is $\big(f(g\omega^j)\big)_{j=0}^{N-1}$, computed by scaling the i-th coefficient of f by g^i and applying the NTT (Subsections 2.2.1 and 2.2.2). In our implementation ω and g are two-power roots of unity obtained from the fixed generator γ, and the codeword is obtained by the scaled NTT of Subsection 2.2.2.

Three properties follow, and together they justify the folding step.

Proposition 4.1. *With D as in Equation (4.3) and $N = 2^k$, $k \geq 1$:*

1. *$\omega^{N/2} = -1$;*
2. *D is closed under negation: $-g\omega^j = g\omega^{j+N/2}$ for $j < N/2$;*
3. *$D^{(2)} = \omega \cdot \langle\omega^2\rangle$, a coset of the subgroup of order $N/2$; in particular $|D^{(2)}| = N/2$.*

Proof. (1) The element $\omega^{N/2}$ squares to $\omega^N = 1$, so it is a square root of unity. It cannot equal 1, since ω has order exactly N; as $\mathbb{F}_p$ is a field, the only square roots of unity are ± 1, hence $\omega^{N/2} = -1$.

(2) Using (1), $g\omega^{j+N/2} = g\omega^j \cdot \omega^{N/2} = -g\omega^j$, and the index $j+N/2$ lies in $\{0, \ldots, N-1\}$, so the negated element is again in D.

(3) Squaring an element of D and applying Equation (4.2) gives $(g\omega^j)^2 = g^2\omega^{2j} = \omega^{2j+1}$, so $D^{(2)} = \{\omega^{2j+1} : j = 0, \ldots, N-1\}$, the subgroup $\langle\omega^2\rangle$ shifted by ω. By (2) the map $x \mapsto x^2$ identifies x with $-x$, so it is two-to-one on D and $|D^{(2)}| = N/2$. □

Property (3) is what makes the construction *self-reproducing*: the squared domain is again a coset of a two-power subgroup, of the same shape as D but half the size, now described by the pair (ω, ω^2) in place of (g, ω). A single FRI round therefore updates the domain simply by squaring both parameters,

$$g \leftarrow g^2, \qquad \omega \leftarrow \omega^2, \tag{4.4}$$

that is, the two domain parameters are squared at the end of each layer. The requirements of Subsection 4.2.1 are preserved at every level, as long as the subgroup order remains even.

Example 4.1. Take $\mathbb{F}_{17}$, whose multiplicative group has order $16 = 2^4$, with generator $\gamma = 3$. For $N = 8$ we get $\omega = 3^2 = 9$ and $g = 3^1 = 3$, so $g^2 = 9 = \omega$ as required by Equation (4.2). The evaluation domain is

$$D = \{3, 10, 5, 11, 14, 7, 12, 6\}.$$

Its elements pair up as $(3, 14)$, $(10, 7)$, $(5, 12)$, $(11, 6)$, each pair summing to 0 mod 17, this is $-g\omega^j = g\omega^{j+4}$ in action. Squaring gives $D^{(2)} = \{9, 15, 8, 2\}$, of size 4, which is exactly the coset $\omega \cdot \langle \omega^2 \rangle = 9 \cdot \langle 13 \rangle$ built from the updated parameters.

4.3 The Folding Operation

Folding is the algebraic core of FRI. In one step it takes a polynomial f of degree less than d, given by its values on the domain D, and produces a new polynomial of degree less than $d/2$, given by its values on the smaller domain $D^{(2)}$. The reduction uses a single random challenge and no interpolation: the values of the new polynomial are a simple combination of the values of the old one. Repeating this step reduces the degree by half each time; the prover stops once the polynomial is small enough to send directly, and transmits its remaining coefficients in the clear.

The section is organised around the three questions that folding answers: how a polynomial splits into two halves (Subsection 4.3.1), how those halves are recovered from evaluations alone (Subsection 4.3.2), and how the challenge combines them into the folded polynomial (Subsection 4.3.3). The two polynomials f_e and f_o introduced next appear throughout the section in three guises. They are first defined as polynomials, then evaluated at a point, then recombined, but they are always the same two objects.

4.3.1 Even-Odd Decomposition

The first step is to split f into the part carried by its even-indexed coefficients and the part carried by its odd-indexed ones. Grouping the coefficients this way lets us write f in terms of two polynomials in X^2, each of roughly half the degree.

Definition 4.2 (Even-Odd Decomposition)**.** Let $f(X) = \sum_{i=0}^{d-1} c_i X^i \in \mathbb{F}_p[X]$. Define

$$f_e(Y) = \sum_{i \text{ even}} c_i \, Y^{i/2}, \qquad f_o(Y) = \sum_{i \text{ odd}} c_i \, Y^{(i-1)/2}.$$

Then f decomposes uniquely as

$$f(X) = f_e(X^2) + X \cdot f_o(X^2), \tag{4.5}$$

and both f_e and f_o have degree less than $\lceil d/2 \rceil$.

In words: f_e collects the even coefficients $c_0, c_2, c_4, \ldots$ and f_o the odd ones $c_1, c_3, c_5, \ldots$, each repackaged as a polynomial in $Y = X^2$. Equation (4.5) just puts them back together, with the extra factor X accounting for the odd powers. Each half keeps at most $d/2$ of the original d coefficients, which is why its degree is halved. This is the same split that drives the recursive step of the NTT (Subsection 2.2.1), and it is the decomposition FRI uses in the commit phase.

4.3.2 Recovering the Components from a Pair

There is a catch: the prover does not hold f as a list of coefficients, but as a list of values on D. So the values of f_e and f_o must be recovered from the values of f alone. The trick is to evaluate f at a point x and at its negative $-x$. Because $(-x)^2 = x^2$, both evaluations land on the same point x^2 of the squared domain, and the odd part flips sign:

$$\begin{aligned} f(x) &= f_e(x^2) + x \cdot f_o(x^2), \\ f(-x) &= f_e(x^2) - x \cdot f_o(x^2). \end{aligned} \tag{4.6}$$

These are two linear equations in the two unknowns $f_e(x^2)$ and $f_o(x^2)$. Adding them isolates the even part; subtracting them isolates the odd part. Since $x \neq 0$ for every $x \in D$, we can divide and obtain

$$f_e(x^2) = \frac{f(x) + f(-x)}{2}, \qquad f_o(x^2) = \frac{f(x) - f(-x)}{2x}. \tag{4.7}$$

So a single pair of values $\big(f(x), f(-x)\big)$ determines both components at the point $x^2 \in D^{(2)}$. By Theorem 4.1 that pair is stored in the codeword at indices i and $i + N/2$, so the prover finds it by reading two entries that sit exactly half the list apart.

4.3.3 The Fold

The final step mixes the two halves using the verifier's random challenge α. The mix collapses f_e and f_o into one polynomial of half the degree.

Definition 4.3 (FRI Fold)**.** Let f have degree less than d and let α be a challenge sampled from a field $\mathbb{K} \supseteq \mathbb{F}_p$. The *fold* of f with respect to α is

$$f^{(\alpha)}(Y) := f_e(Y) + \alpha \cdot f_o(Y), \tag{4.8}$$

a polynomial over $\mathbb{K}$ of degree less than $\lceil d/2 \rceil$.

Putting Equation (4.8) together with Equation (4.7), the values of $f^{(\alpha)}$ on $D^{(2)}$ come straight from the values of f on D, with no interpolation anywhere:

$$f^{(\alpha)}(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x}. \tag{4.9}$$

This is exactly what our implementation does: for each $i < N/2$ it reads the two entries at indices i and $i + N/2$, applies Equation (4.9) with $x = g\omega^i$, and writes out a codeword of length $N/2$ on the domain $D^{(2)}$ given by the updated parameters of Equation (4.4).

One round thus halves two things at once: the degree bound, by Definition 4.3, and the domain size, by Theorem 4.1. The rate ρ stays the same, so the folded codeword is again a Reed-Solomon codeword of the same rate, which is what lets the next round repeat the identical step. The prover keeps folding until the degree is small enough to send the remaining polynomial directly.

Remark 4.2 (Arithmetic in the extension field). The initial codeword has entries in $\mathbb{F}_p$, but the challenge α is sampled from the quartic extension $\mathbb{F}_{p^4}$ (Subsection 2.1.3). Because Equation (4.8) multiplies f_o by α, the folded polynomial already has coefficients in $\mathbb{F}_{p^4}$, and so does every layer after it. In our implementation this switch happens at the very first fold: the initial codeword holds base-field elements, every later layer holds extension-field elements, and the Merkle leaves grow accordingly from four to sixteen bytes.

Example 4.2. We continue Example 4.1. Let $f(X) = 1 + 2X + 3X^2 + 4X^3$ over $\mathbb{F}_{17}$, so $d = 4$, $N = 8$, and $\rho = 1/2$. Evaluating f on $D = \{3, 10, 5, 11, 14, 7, 12, 6\}$ gives the codeword

$$\big(f(x)\big)_{x \in D} = (6, 3, 8, 15, 16, 4, 8, 16).$$

Splitting f as in Definition 4.2 gives $f_e(Y) = 1 + 3Y$ and $f_o(Y) = 2 + 4Y$. With the challenge $\alpha = 5$, the fold is

$$f^{(5)}(Y) = (1 + 3Y) + 5\,(2 + 4Y) = 11 + 6Y,$$

of degree $1 < 2$, exactly as Definition 4.3 predicts. Applying Equation (4.9) to the four pairs $\big(f(x), f(-x)\big)$ gives the codeword $(14, 16, 8, 6)$ on $D^{(2)} = \{9, 15, 8, 2\}$, which is precisely the evaluation of $11 + 6Y$ on that domain. Both the degree bound and the domain size have halved, and the rate is still $2/4 = 1/2$.

4.4 The Protocol

Section 4.3 Section 4.3 defined the folding operation. The protocol embeds it in an interaction between prover and verifier. It has two phases. In the *commit phase* the prover folds the polynomial down to a short final layer, committing to each intermediate codeword with a Merkle tree so that it cannot change its mind later. In the *query phase* the verifier picks a few random positions and asks the prover to reveal the values needed to re-check that each fold was done correctly. If the prover cheated, that is, if the starting codeword was not close to a low-degree polynomial, then at least one of these checks fails with high probability.

Throughout, $D_0 = D$ is the initial domain of size N, and D_r is the domain after r folds, of size $N/2^r$.

4.4.1 Commit Phase

The prover starts from the coefficients of f, extends them to a codeword on D by the low-degree extension (Subsection 2.2.2), and then repeats the same three actions at every round: commit, receive a challenge, fold.

1. **Commit.** Send the oracle for the current codeword. This binds the prover to that exact codeword: every later query must be answered consistently with it.

2. **Challenge.** Obtain the folding challenge α_r from the verifier (once made non-interactive by the BCS transformation of Section 3.3, this challenge is derived from the transcript).

3. **Fold.** Apply Equation (4.9) with α_r to halve the codeword, and square the two domain parameters (Equation (4.4)) to move to D_{r+1}.

The process stops once the codeword has shrunk to a short final layer, which the prover sends directly. The number of rounds is $\log_2 N$ down to a constant, or $\log_2(N/\ell)$ if the prover stops at a final layer of length ℓ.

In the IOP the prover sends, at each round, the oracle for the current codeword, and the verifier replies with the folding challenge; the concrete realisation of these oracles as commitments follows the BCS transformation of Section 3.3. For a polynomial of length 8 with blowup 2, so $N = 16$, the process runs $\log_2 N = 3$ rounds down to a constant final layer.

4.4.2 Query Phase

Committing to the codewords is not enough on its own: the verifier must check that consecutive layers are actually related by the fold. It does so by sampling a small number of random positions and, at each one, asking the prover for just the values needed to recompute one fold.

For a single query the verifier picks an index and follows it down through the layers. At round r, with domain size n_r, the index determines a pair of positions half the list apart,

$$\mathrm{lo} = \mathrm{idx} \bmod (n_r/2), \qquad \mathrm{hi} = \mathrm{lo} + n_r/2,$$

which are exactly the entries $f(x)$ and $f(-x)$ that Equation (4.9) combines. The prover returns both values together with their Merkle authentication paths. The index for the next round is then $\mathrm{idx} \leftarrow \mathrm{lo}$, so one random choice in the first layer determines the whole chain of positions down to the final layer.

In IOP terms, the verifier samples a number of indices in $[0, N/2)$ and, for each one, queries the two oracle positions per layer needed to recompute one fold. Each query thus costs one pair of oracle queries per round.

4.4.3 Verification and the Colinearity Check

The verifier holds only the proof, namely the Merkle roots, the final layer, and the opened values with their paths, and must decide whether to accept. It proceeds in three steps.

First, it **reconstructs the challenges**. Re-reading the roots and the final layer from the transcript in the same order the prover wrote them, it recomputes every α_r and every domain parameter, and checks that the number of rounds matches the expected $\log_2(N/\ell)$.

Second, for each queried position it **checks the Merkle paths**, confirming that the two opened values really sit at their claimed positions under the committed root. A failed path means the prover is opening something it did not commit to.

Third, it performs the **colinearity check**. Using the opened pair $(\mathrm{lo}, \mathrm{hi})$ at round r, it recomputes the folded value by the same rule the prover should have used,

$$v = \frac{\mathrm{lo} + \mathrm{hi}}{2} + \alpha_r \cdot \frac{\mathrm{lo} - \mathrm{hi}}{2\,x}, \qquad x = \mathrm{offset}_r \cdot \omega_r^{\mathrm{lo}}, \tag{4.10}$$

and compares v against the value at the corresponding position of the *next* layer, which is an opened entry for intermediate rounds, or the final layer for the last round. If the two disagree, the fold was not performed honestly and the verifier rejects.

The name *colinearity check* comes from the geometry of Equation (4.10). The two points (x, lo) and $(-x, \mathrm{hi})$, together with the folded value at α_r, must lie on a single line, since the line through $f(x)$ and $f(-x)$ evaluated at α_r is exactly $f^{(\alpha_r)}(x^2)$. Each query verifies one such local relation per layer; a codeword far from low degree cannot satisfy all of them at once, so a handful of random queries suffices to catch a cheating prover.

In IOP terms the verifier reconstructs the challenges from the transcript, checks that each queried oracle answer is consistent (which, after compilation, means checking a commitment opening), and verifies the colinearity relation Equation (4.10) at every queried position of every layer.

4.5 Soundness

The earlier sections describe an honest prover. Soundness asks the opposite: if the committed function is *not* close to any low-degree polynomial, can a cheating prover still make the verifier accept? We show the answer is no, except with small probability, following the round-by-round analysis of Garreta, Mohnblatt, and Wagner [GMW25], which gives the simplest known soundness proof for FRI in the form implemented here.

The key observation is the following. FRI certifies *proximity*, not equality (Remark 4.1): the verifier never learns that the committed vector is exactly a codeword, only that it is close to one. Soundness must therefore be stated in terms of distance. We fix a distance parameter δ and ask that, if the initial function is δ-far from the code $\mathrm{RS}[\mathbb{F}, D, \rho]$ in relative Hamming distance (Definition 2.8), then the verifier rejects with high probability. The difficulty is that the protocol only ever inspects a few points, so we must argue that being far from the code cannot quietly disappear during folding, and that it is caught by the queries.

4.5.1 The Two Things That Can Go Wrong

A cheating prover has exactly two ways to succeed, and the proof bounds each in turn.

The first is that *folding rescues a far function*. Each round replaces the current layer f_i by $f_{i+1} = f_{i,e} + \alpha_i f_{i,o}$, a random linear combination of its two halves. If some challenge α_i made f_{i+1} close to the code even though f_i was far, the prover would have folded its way from a dishonest layer to an honest-looking one, and no amount of querying would detect it. So we need folding to *preserve distance*: for all but a few unlucky α_i, a far layer stays far.

The second is that *the queries all miss*. Even if every layer stays far, the verifier only checks a few random positions. If the far parts of the final layer happened to avoid every queried position, the cheat would slip through. So we need enough queries that missing all of them is unlikely.

The next two subsections address these in order: first the algebraic fact that makes folding preserve distance, then the theorem that combines it with the query bound.

4.5.2 Why Folding Preserves Distance

Distance preservation is not obvious, precisely because f_{i+1} is a chosen linear combination of $f_{i,e}$ and $f_{i,o}$, and one might fear a lucky α_i that makes it close to the code by accident. The property that rules this out is *correlated agreement*, introduced in the study of proximity gaps [BS+20]: if a random combination of several functions is close to the code, then those functions are not just individually close, they agree with codewords on a *common* set of positions.

The round-by-round proof uses a sharper version, *mutual* correlated agreement, introduced in the analysis of WHIR [Arn+24]. It requires that the set of positions where the combination agrees with the code is the *same* set on which the two halves agree. This equality of sets is what lets the argument repeat cleanly at every round: the agreement pattern of one layer passes to the next without drift, so distance is tracked faithfully all the way down. This is why we use mutual correlated agreement rather than the older notion.

Crucially, mutual correlated agreement is a *theorem*, not an assumption, as long as the distance stays below the Johnson radius $1 - \sqrt{\rho}$ (Theorem 2.1) [GMW25]. This is the regime in which the soundness of our implementation is established, and we return to what lies beyond it at the end of the section.

4.5.3 The Soundness Theorem

With distance preservation in hand, the two risks of Subsection 4.5.1 become two error terms.

Theorem 4.2 (Round-by-round soundness of FRI, informal [GMW25, Theorem 5.2])**.** *Let f be δ-far from $\mathrm{RS}[\mathbb{F}, D, \rho]$ with δ below the Johnson radius $1 - \sqrt{\rho}$. Then any cheating prover makes the verifier accept with probability at most*

$$\varepsilon_{\mathrm{FRI}} \;=\; \underbrace{\frac{r}{|\mathbb{K}|} \cdot \mathrm{poly}(|D|)}_{\textit{folding}} \;+\; \underbrace{(1-\delta)^s}_{\textit{queries}},$$

where $\mathbb{K}$ is the field the challenges are drawn from, $r = \log_2(|D|/\ell)$ the number of rounds, and s the number of queries.

Proof sketch. The proof tracks one bit of information through the protocol: whether the current layer is still far from its code. Following [GMW25] we call the state *doomed* while this holds. The argument is that a doomed state stays doomed through folding, and that a doomed final state is caught by the queries.

The start is doomed. By assumption f_0 is δ-far from the code, so the state is doomed before any interaction. The prover has committed to a far vector and cannot take it back.

Folding keeps it doomed ([GMW25, Lemma 5.1]). Suppose layer f_i is δ-far. Its fold is $f_{i+1} = f_{i,e} + \alpha_i f_{i,o}$. If f_{i+1} were close to the code for many values of α_i, mutual correlated agreement (Subsection 4.5.2) would force $f_{i,e}$ and $f_{i,o}$ to agree with codewords on a common set large enough to make f_i itself close, which contradicts the assumption. Hence only a few α_i are bad, a fraction of about $1/|\mathbb{K}|$ of the field. A

union bound over the r rounds gives the folding term. This is the only place the field size enters, and it fixes a requirement on $\mathbb{K}$: the quantity $1/|\mathbb{K}|$ must be negligible, so $\mathbb{K}$ must be large enough in absolute terms. The base field is in general too small for this, and the challenges are therefore drawn from a sufficiently large extension. The concrete choice of extension is an implementation matter, discussed in Chapter 6.

The queries catch a doomed final layer ([GMW25, Claim 5.3]). If every fold kept the state doomed, the final layer sent in the clear is itself δ-far from a low-degree polynomial. A single query picks a random position and checks the colinearity relation Equation (4.10) along the chain; since the final layer disagrees with the code on at least a δ fraction of positions, the query exposes a broken fold with probability at least δ. The s queries are independent, so all of them miss with probability at most $(1-\delta)^s$, the query term.

The prover wins only if some fold got lucky or every query missed, so the total is the sum of the two terms. □

The two terms are controlled by different means, which is what makes the protocol practical. The folding term is negligible for free: with $\mathbb{K} = \mathbb{F}_{p^4}$ of size about 2^{124} (Remark 4.2) and only a handful of rounds, it sits around 2^{-122}. The query term is the one the designer tunes, by choosing how many queries to make.

Example 4.3. Take a final layer that disagrees with the code on half its positions, $\delta = 1/2$, and $s = 20$ queries. One query misses with probability $1/2$, so twenty independent queries all miss with probability $(1/2)^{20} \approx 10^{-6}$, about 20 bits of security from the query phase. A smaller rate raises the detection probability per query and so buys more bits each. The concrete parameters used in this work, and the resulting security level, are discussed in Chapter 6.

4.5.4 Beyond the Provable Regime

Everything above lives below the Johnson radius $1 - \sqrt{\rho}$, where mutual correlated agreement is proved. Pushing the analysis past this radius, towards the list-decoding *capacity* $1-\rho$, is an active line of research: recent work extends mutual correlated agreement beyond the Johnson radius for Reed-Solomon codes, narrowing the gap between what is provable and what was previously only conjectured, and the question is the subject of a dedicated open-problem effort [Eth24]. Approaching capacity would allow fewer queries for the same security, though it is known that the list-decoding capacity itself cannot be achieved. In this thesis we rely only on the established Johnson-radius guarantee, and leave the capacity regime out of scope

Chapter 5

FRI as a Polynomial Commitment Scheme

The FRI protocol of Chapter 4 tests one thing: that a committed function is close to a low-degree polynomial. On its own this is a proximity test, not a commitment scheme. A polynomial commitment scheme (Section 2.7) must do more: it must let the prover commit to a polynomial f and later prove, at any point z chosen by the verifier, that $f(z) = v$ for a claimed value v. This chapter shows how the proximity test of FRI is turned into such an evaluation proof through a single algebraic idea, the quotient technique, and how the four algorithms of a polynomial commitment scheme are realised on top of it.

5.1 From Low-Degree Testing to Evaluation Proofs

Recall the gap between what FRI provides and what a commitment scheme needs. FRI convinces the verifier that the committed codeword is close to some polynomial of degree less than d, but says nothing about the value of that polynomial at any particular point. FRI asserts a global property: that the committed word is close to some low-degree polynomial, without naming it or any of its values, an evaluation proof fixes a specific point z and a specific value v and asserts that $f(z) = v$ for the committed polynomial.

The bridge between the two is the observation that an evaluation claim can be rephrased as a low-degree claim about a different polynomial. If $f(z) = v$, then $f(X)-v$ has a root at z, so it is divisible by $X - z$ and the quotient is again a polynomial. If instead $f(z) \neq v$, the division leaves a remainder and the quotient is not a polynomial at all. Testing whether that quotient is a low-degree polynomial, which is exactly what FRI does, therefore tests whether the evaluation claim is true.

5.2 The Quotient Technique

Let f be a polynomial of degree less than d, let z be an evaluation point, and let v be a claimed value. Define the *quotient*

$$q(X) = \frac{f(X) - v}{X - z}. \tag{5.1}$$

The whole construction rests on the following elementary fact.

Proposition 5.1. *The quotient $q(X)$ of Equation (5.1) is a polynomial of degree less than $d-1$ if and only if $f(z) = v$.*

Proof. By the factor theorem, $X - z$ divides a polynomial g exactly when $g(z) = 0$. Applying this to $g(X) = f(X) - v$: if $f(z) = v$ then $g(z) = 0$, so $X - z$ divides g and $q = g/(X - z)$ is a polynomial, of degree one less than g, hence less than $d - 1$. Conversely, if $f(z) \neq v$ then $g(z) \neq 0$, the division of g by $X - z$ leaves a nonzero remainder, and q is not a polynomial. □

Remark 5.1 (Opening at several points). The same idea can be used to prove several evaluations at once. To open f at points $z_1, \ldots, z_k$ with claimed values $v_1, \ldots, v_k$, let r be the unique polynomial of degree less than k with $r(z_j) = v_j$ for all j, obtained by interpolation. Then $f(X) - r(X)$ vanishes at every z_j exactly when all the claims hold, so

$$q(X) = \frac{f(X) - r(X)}{\prod_{j=1}^{k}(X - z_j)}$$

is a polynomial if and only if $f(z_j) = v_j$ for every j, and a single FRI proof on q certifies all k evaluations. The single-point case of Equation (5.1) is $k = 1$, where r is the constant v.

The proposition reduces certifying an evaluation to a low-degree test. The prover compute the quotient q and runs FRI on it. If the claim $f(z) = v$ holds, q is a polynomial of degree less than $d-1$ and FRI accepts. If the claim is false, q has a pole at z and its evaluation vector lies far from every low-degree codeword, so FRI rejects with high probability by the soundness of Section 4.5. The intuition is that a false value cannot be hidden: it turns the quotient from a low-degree polynomial into one whose distance from the code the FRI queries detect. The evaluation proof is therefore a FRI proof about the quotient rather than about f itself.

For the quotient to be well defined on the evaluation domain, the point z must lie outside that domain: if z were a domain point, the factor $X - z$ would vanish there and q could not be evaluated at that point. This is a genuine limitation of the scheme. A polynomial commitment scheme should open at any point of the field, whereas here the domain points are excluded, so the construction is a commitment that opens everywhere except on the domain. The gap is harmless in use: the domain is a small structured subset of the field, and the evaluation points that occur in practice, such as the random challenges drawn by an outer protocol, fall outside it with overwhelming probability.

Example 5.1. Take $f(X) = 1 + 2X + 3X^2 + 4X^3$ over $\mathbb{F}_{17}$ and the point $z = 5$, for which $f(5) = 8$. With the correct value $v = 8$,

$$f(X) - 8 = 4X^3 + 3X^2 + 2X + 10 = (X - 5)(4X^2 + 6X + 15),$$

so $q(X) = 4X^2 + 6X + 15$ is a polynomial of degree $2 < 3$, and FRI on q succeeds. If instead the prover claims the wrong value $v' = 9$, then $f(X) - 9$ is not divisible by $X - 5$: the division leaves a nonzero remainder, the quotient is not a polynomial, and FRI on it fails.

5.3 Commit, Open, and Verify

The quotient technique supplies the evaluation proof; wrapping it with a commitment to f gives the four algorithms of a polynomial commitment scheme (Section 2.7). Because the scheme is built on FRI, it is transparent: the setup consists only of public parameters, with no trusted secret.

Setup. Fix the public parameters: the field, the evaluation domain D (Section 4.2), the rate ρ, and the degree bound d. No trusted setup is required, since these are public and verifiable.

Commit. To commit to f, evaluate it on the domain D by the low-degree extension and send the oracle for the resulting codeword, that is, its Merkle commitment (Chapter 3). The commitment C is the Merkle root.

Open. To prove $f(z) = v$, the prover forms the quotient q of Equation (5.1), encodes it into a codeword and commits to it to its codeword, and produces a FRI proof that q has degree less than $d - 1$. The opening consists of the claimed value v, together with this FRI proof and the query openings that tie q back to the committed f at the sampled positions.

Verify. The verifier checks the FRI proof for q and, at each queried position x, checks the algebraic relation that links the two codewords,

$$q(x) \cdot (x - z) = f(x) - v, \tag{5.2}$$

using the opened values of f and q at x against their committed roots. If the FRI proof verifies and Equation (5.2) holds at every queried position, the verifier accepts $f(z) = v$.

The soundness of the scheme follows from that of FRI. A prover who claims a false value $v \neq f(z)$ is forced, by Theorem 5.1, to commit to a quotient that is not a low-degree polynomial; the FRI test on q then rejects except with the small probability of Theorem 4.2. The consistency check Equation (5.2) prevents the prover from committing to an unrelated low-degree polynomial in place of the true quotient, since that relation must hold at every queried point against the commitment to f.

Example 5.2. We follow the polynomial of Example 5.1, $f(X) = 1+2X+3X^2+4X^3$ over $\mathbb{F}_{17}$, and use the evaluation domain $D = \{3, 10, 5, 11, 14, 7, 12, 6\}$ of Example 4.1. Since the point $z = 5$ of Example 5.1 lies in D, for a full opening we take a point outside the domain, $z = 2$, with $v = f(2) = 15$.

Commit. The prover evaluates f on D, giving the codeword $(6, 3, 8, 15, 16, 4, 8, 16)$, and sends its Merkle root as the commitment C.

Open. To prove $f(2) = 15$, the prover forms the quotient $q(X) = (f(X)-15)/(X-2) = 7 + 11X + 4X^2$, of degree $2 < 3$, evaluates it on D, and produces a FRI proof that q is low-degree, opening f and q at the same query positions.

Verify. At each queried point the verifier checks Equation (5.2). For example, at $x = 10$ the opened values are $f(10) = 3$ and $q(10) = 7$, and indeed

$$q(10)\,(10-2) = 7 \cdot 8 = 5 = 3 - 15 = f(10) - v \pmod{17},$$

so the check passes. Had the prover claimed the wrong value $v' = 16$, the quotient $(f-16)/(X-2)$ would not be a polynomial, FRI would reject it as far from low degree, and the relation would fail: at $x = 10$ it would ask for $5 = 3 - 16 = 4$, which is false.

Chapter 6

Implementation

The previous chapters describe FRI and the polynomial commitment scheme in abstract terms, using oracles and queries. This chapter describes how they are actually implemented in code. We present the structure of the implementation, the cryptographic building blocks, the optimisations added to the basic protocol, and the concrete parameters and the security level they give.

The implementation is written in Python and uses no external cryptographic libraries: the field arithmetic, the hash function, the Merkle commitments, and the Fiat-Shamir challenger are all written from scratch.

Python fits the aim of this implementation, which is clarity rather than speed.

6.1 Architecture

The code is split into modules, each responsible for one part of the system.

- `field.py` defines the BabyBear prime field: modular arithmetic, inversion by Fermat's little theorem, and the two-adic generators used to build evaluation domains.
- `extension.py` defines the quartic extension $\mathbb{F}_{p^4}$ over the irreducible polynomial $X^4 - 11$, from which the folding challenges are drawn.
- `polynomial.py` provides the Number Theoretic Transform, the coset low-degree extension, and polynomial evaluation.
- `keccak.py` implements the Keccak-$f[1600]$ permutation.
- `merkle_tree.py` builds Merkle trees over codewords and produces and checks authentication paths.
- `challenger.py` implements the duplex-sponge Fiat-Shamir transcript on top of the Keccak permutation.
- `fri.py` ties the above together into the commit phase, the query phase, and verification.

These modules map onto the theory: `field`, `extension` and `polynomial` realise the algebraic objects of Chapter 2; `merkle_tree` and `challenger` realise the BCS transformation of Section 3.3; and `fri` realises the protocol of Chapter 4.

6.2 Cryptographic Components

Two roles are served by hashing, and the implementation uses a different primitive for each, as anticipated in Section 2.5: SHA-256 for the Merkle commitments, where only collision resistance is needed, and a Keccak-based duplex sponge for the Fiat-Shamir challenger.

6.2.1 Merkle Commitments

The oracles of the FRI IOP are realised as Merkle vector commitments. For each layer the prover hashes every codeword entry into a leaf with SHA-256, builds the binary tree, and publishes the root. A query at a position is answered with the entry together with its authentication path, and the verifier recomputes the root as in Figure 3.1. This is exactly the "oracles become commitments, queries become openings" mechanism of the BCS transformation.

6.2.2 The Duplex-Sponge Challenger

The Fiat-Shamir challenges are produced by a challenger that maintains a transcript and derives every challenge from it, so that the verifier can reproduce the same challenges offline. The construction used is a *duplex sponge*. Chiesa and Orrù [CO25] show that a duplex sponge efficiently supports the Fiat–Shamir transformation through its native absorb and squeeze operations, which is why the challenger uses Keccak rather than SHA-256.

The sponge keeps a state that is split into a public *rate*, into which data is absorbed and from which output is squeezed, and a hidden *capacity*, which carries the security (Figure 6.1). Concretely the state is the 1600-bit Keccak state, viewed as 25 lanes of 64 bits; the first 17 lanes (136 bytes) are the rate and the remaining 8 lanes (64 bytes) the capacity. Absorbing XORs bytes into the rate and applies the permutation when the rate fills; squeezing reads bytes back out, permuting again when the rate is exhausted.

The transcript is written and read in a fixed order, which the verifier must reproduce exactly: the parameters are absorbed first, then for each round the Merkle root is absorbed and the folding challenge squeezed, and after the last fold the final layer is absorbed; the query indices are squeezed once the whole commit transcript is fixed. Two kinds of challenge are drawn from the squeezed bytes: the folding challenges, which are elements of $\mathbb{F}_{p^4}$ obtained from sixteen squeezed bytes read as four base-field coefficients, and the query indices, obtained by reducing squeezed bytes modulo the domain size. The whole Keccak permutation is implemented from scratch, so the challenger depends on no external library.

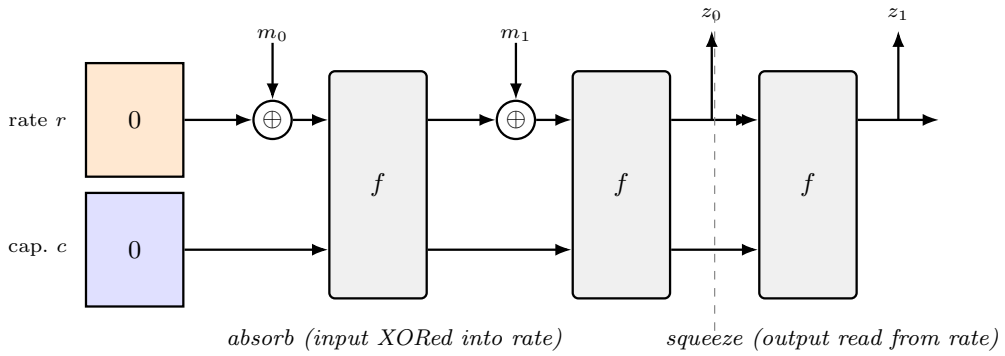


Figure 6.1: The duplex sponge. In the *absorb* phase each input block m_i is XORed into the rate and the state is passed through the permutation f; in the *squeeze* phase output blocks z_i are read from the rate, applying f again between reads. The capacity is never read from or written to directly, which is what carries the security of the construction.

6.3 Optimisations

The base protocol of Chapter 4 admits several standard optimisations. They do not change what the protocol proves; they reduce proof size or prover work.

Early stopping of the final layer. Rather than folding all the way down to a single constant, the prover stops once the codeword reaches a small fixed length and sends the whole remaining layer in the clear. This is governed by the parameter `final_poly_len`: folding continues while the codeword is longer than `final_poly_len`. The choice trades rounds against the size of the final layer. For a polynomial of 8 coefficients with blowup 2 the initial codeword has $N = 16$ entries, and the options are:

`final_poly_len`	rounds	Merkle roots	final values sent
1	4	4	1
2	3	3	2
4	2	2	4
8	1	1	8

Each step up the target length removes one round, and with it one Merkle commitment and its query openings, at the cost of doubling the number of values sent in the clear at the end. The value `final_poly_len` $= 2$ used here keeps most of the rounds while sending only two final values.

Proof-of-work grinding. A second, standard optimisation is *grinding*, which buys security without adding queries [Sta21, Section 3.11.3]. Before the query positions are fixed, the prover must find a *nonce* whose hash begins with g zero bits, costing about 2^g hash attempts. An honest prover does this once; a cheating prover, whose attack is a retry loop that needs a fresh transcript, and hence a fresh nonce, at every attempt, pays it each time. This multiplies the attacker's effort by 2^g, adding g bits on top of

the query term:

$$\lambda \;=\; \underbrace{s \cdot \tfrac{1}{2}\log_2(1/\rho)}_{\text{from the queries}} \;+\; \underbrace{g}_{\text{from grinding}} \quad . \tag{6.1}$$

Here λ is the security level in bits, related to the soundness error $\varepsilon_{\mathrm{FRI}}$ of Theorem 4.2 by $\lambda = -\log_2 \varepsilon_{\mathrm{FRI}}$: each added bit halves the probability that a cheating prover is accepted. This optimisation is present in our implementation only as a stub, with an empty proof-of-work routine and the difficulty fixed to zero, so it adds nothing here ($g = 0$).

6.4 Parameters and Security Level

The soundness bound of Theorem 4.2 turns the choice of parameters into a choice of security level. Its two error terms are the folding term $\frac{r}{|\mathbb{K}|} \cdot \mathrm{poly}(|D|)$, which depends on the size of the challenge field, and the query term $(1-\delta)^s$, which depends on the number of queries s and the distance δ. Because challenges are drawn from the quartic extension $\mathbb{F}_{p^4}$, with $|\mathbb{K}| = p^4 \approx 2^{124}$, the folding term is at most about 2^{-122} for the round counts used here and is never the bottleneck. The security level is therefore set by the query term.

In the provable regime the distance is bounded by the Johnson radius $\delta < 1 - \sqrt{\rho}$ (Theorem 2.1), and each query contributes

$$-\log_2(1-\delta) \;=\; \tfrac{1}{2}\log_2(1/\rho) \tag{6.2}$$

bits of security, so s queries give $\frac{s}{2}\log_2(1/\rho)$ bits. The rate ρ thus controls how much each query is worth: a smaller rate (a larger blowup factor) buys more bits per query, at the cost of a larger evaluation domain and more prover work.

Since grinding is not enabled in our configuration (Section 6.3), the security level here comes entirely from the queries; the early-stopping optimisation does not affect it, as it only trades rounds against the length of the final layer and so changes the proof size rather than the security level.

Parameters used in this work. The configuration evaluated in this work is summarised in table 6.1. `final_poly_len` sets the early-stopping point and `proof_of_work_bits` the grinding difficulty of Section 6.3. The rate is $\rho = 1/2$, so by Equation (6.2) each query is worth 0.5 bits and the 20 queries provide about 10 bits; grinding is set to 0, so by Equation (6.1) it adds nothing and the security comes entirely from the queries. This is deliberately a *test* configuration, chosen so that proofs are small and fast to generate while exercising every part of the protocol; it is not a production security level.

Here `final_poly_len` sets the early-stopping point and `proof_of_work_bits` the grinding difficulty of Section 6.3. The rate is $\rho = 1/2$, so by Equation (6.2) each query is worth 0.5 bits and the 20 queries provide about 10 bits; grinding is set to 0, so by Equation (6.1) it adds nothing and the security comes entirely from the queries. This is deliberately a *test* configuration, chosen so that proofs are small and fast to generate while exercising every part of the protocol; it is not a production security level.

Parameter	Value	Meaning
`log_blowup`	1	rate $\rho = 1/2$
`num_queries`	20	number of query repetitions
`proof_of_work_bits`	0	grinding disabled
`final_poly_len`	2	early-stopping length

Table 6.1: Parameters of the evaluated configuration.

Reaching a production level. The security level is raised by lowering the rate, which increases the bits per query: at $\rho = 1/4$ each query is worth 1 bit, and at $\rho = 1/8$ it is worth 1.5 bits. A larger blowup makes each query more powerful, but it enlarges the committed codeword, so in practice a small blowup of 2 or 4 is usually preferred, reaching the target with more queries rather than a very low rate. The number of queries needed to reach a target of λ bits is obtained by inverting Equation (6.2):

$$s \;=\; \frac{\lambda}{\frac{1}{2}\log_2(1/\rho)}. \tag{6.3}$$

For $\lambda = 100$ this gives $100/0.5 = 200$ queries at $\rho = 1/2$, $100/1 = 100$ at $\rho = 1/4$, and $100/1.5 \approx 67$ at $\rho = 1/8$.

	blowup	ρ	bits/query	queries for 100 bits
$\rho = 1/2$	2	1/2	0.5	≈ 200
$\rho = 1/4$	4	1/4	1.0	≈ 100
$\rho = 1/8$	8	1/8	1.5	≈ 67

The query counts in the last column assume no grinding; enabling grinding of g bits would lower each of them by g divided by the bits per query. With a larger blowup each query is more effective, so fewer repetitions are needed. The parameters above target correctness testing, not deployment security.

6.5 The Polynomial Commitment Scheme

The polynomial commitment scheme of Chapter 5 is implemented in `pcs.py` as a thin layer over the FRI prover and verifier. It exposes the three operations `commit`, `open_at` and `verify`, and reuses the FRI machinery for the low-degree test on the quotient.

`commit` evaluates f on the low-degree extension coset, hashes each entry into a Merkle leaf with SHA-256, and returns the root together with the prover's copy of the codeword and tree.

`open_at` implements the quotient technique. It first rejects any evaluation point z that lies in the coset domain: since the domain is a coset $g\langle\omega\rangle$ with $g^N = -1$, the check is $z^N \neq g^N$, and if it fails the quotient would require dividing by zero at a domain point. It then forms the quotient coefficients of Equation (5.1) by the synthetic division

$$q_{d-1} = a_d, \qquad q_{i-1} = a_i + z\,q_i \quad (i = d-1, \ldots, 1),$$

where $a = f - v$, runs the FRI prover on q, and opens the committed f at the same query positions FRI used on q. Those positions are recovered by replaying the Fiat-Shamir transcript exactly as the verifier will, so both sides agree on the same indices without further communication.

`verify` runs the FRI verifier on the quotient proof, then at each queried position x checks the consistency relation in the form used by the code,

$$q(x)\,(x - z) = f(x) - v, \tag{6.4}$$

which is the check of Equation (5.2), using the opened value of f checked against the commitment root and the value of q taken from the first FRI layer. Acceptance requires the FRI proof to verify and Equation (6.4) to hold at every queried position.

6.6 Evaluation

To make the parameter trade-offs concrete, we measured the implementation on a polynomial of 256 coefficients. All figures are for the provable regime with grinding disabled, so a target of λ bits fixes the number of queries as $s = \lambda / \left(\frac{1}{2}\log_2(1/\rho)\right)$.

Table 6.2 reports the proof size for three rates and three security targets. For a fixed rate the proof grows linearly with the target, since it grows linearly with the number of queries; for a fixed target it shrinks as the rate decreases, because a smaller rate makes each query carry more bits and fewer are needed.

	$\lambda = 80$	$\lambda = 100$	$\lambda = 128$
$\rho = 1/2$ (blowup 2)	480 KB	600 KB	768 KB
$\rho = 1/4$ (blowup 4)	293 KB	366 KB	468 KB
$\rho = 1/8$ (blowup 8)	237 KB	293 KB	377 KB

Table 6.2: Proof size for a 256-coefficient polynomial, at three rates and three security targets.

Table 6.3 isolates the trade-off at a fixed target of 100 bits. Lowering the rate reduces the proof size, but raises the prover time, because the committed codeword is larger and both the low-degree extension and the Merkle tree grow with it. This is why a small blowup is often preferred in practice: it keeps the prover cheap, at the cost of more queries and a larger proof.

	blowup	queries	prover time	proof size
$\rho = 1/2$	2	200	43 ms	600 KB
$\rho = 1/4$	4	100	51 ms	366 KB
$\rho = 1/8$	8	67	86 ms	293 KB

Table 6.3: Proof size against prover time at a fixed security level of 100 bits, for a 256-coefficient polynomial. Times are wall-clock medians of the Python implementation and are indicative of the relative trend rather than of absolute performance.

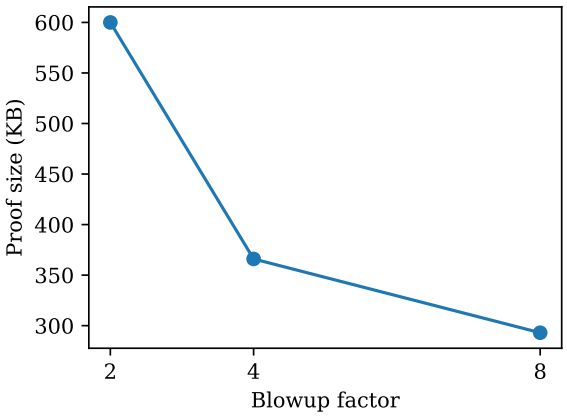


Figure 6.2: Proof size as a function of the blowup factor at 100-bit security for a 256-coefficient polynomial.

6.7 Testing

Correctness is checked by two small test suites. The FRI suite in `test_fri.py` runs an honest prover end to end and accepts its proof, then constructs two dishonest provers, one corrupting a committed value and one corrupting the final layer, and confirms that verification rejects both, so the colinearity checks catch the inconsistencies. It also checks the Number Theoretic Transform against direct evaluation of the same polynomial, verifying the coefficient-to-evaluation conversion. The commitment-scheme suite in `test_pcs.py` covers the evaluation proof: an honest opening at a point outside the domain is accepted, while a claim with a wrong value and an opening with a tampered f value are both rejected.

Chapter 7

Conclusions

This thesis presented a FRI-based polynomial commitment scheme over the BabyBear field, developed from its mathematical foundations to a complete, working implementation. Starting from finite fields and Reed-Solomon codes, we described the interactive oracle proof model and its compilation through the BCS transformation, the FRI protocol and its soundness, and the quotient technique that turns FRI into a full evaluation proof. Every component was implemented from scratch in Python, and the scheme was validated by a test suite covering both the low-degree test and the evaluation proof.

The result is a compact and readable realisation of a transparent, hash-based commitment scheme, in which each part of the construction can be followed directly in the code.

Several directions remain open for future work. The proof-of-work grinding step, prepared in the parameters but left as a stub, would raise the security level at negligible verifier cost. Beyond this, the polynomial commitment scheme is the key ingredient of a full STARK: adding an algebraic intermediate representation and the constraint machinery on top of it would turn the present system into a complete proof system for general computations. Finally, porting the implementation to a compiled language and adding the standard batching and low-level optimisations would bring its performance closer to that of production systems.

Bibliography

[Arn+24] Gal Arnon et al. *WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification*. Cryptology ePrint Archive, Paper 2024/1586. https://eprint.iacr.org/2024/1586. 2024.

[BS+18] Eli Ben-Sasson et al. *Scalable, Transparent, and Post-Quantum Secure Computational Integrity*. Cryptology ePrint Archive, Report 2018/046. https://eprint.iacr.org/2018/046. 2018.

[BS+20] Eli Ben-Sasson et al. "Proximity Gaps for Reed-Solomon Codes". In: *Proceedings of the 61st IEEE Symposium on Foundations of Computer Science (FOCS)*. 2020.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. "Interactive Oracle Proofs". In: *Theory of Cryptography Conference (TCC)*. IACR ePrint 2016/116. https://eprint.iacr.org/2016/116. 2016.

[CO25] Alessandro Chiesa and Eylon Orrù. *Fiat-Shamir via Sponges*. Cryptology ePrint Archive, Report 2025/536. https://eprint.iacr.org/2025/536. 2025.

[Eth24] Ethereum Foundation. *The RS Proximity Gaps Prize*. https://proximityprize.org. 2024.

[GMW25] Albert Garreta, Nicolas Mohnblatt, and Benedikt Wagner. *A Simplified Round-by-round Soundness Proof of FRI*. Cryptology ePrint Archive, Paper 2025/1993. https://eprint.iacr.org/2025/1993. 2025.

[HASW24] Mathias Hall-Andersen, Mark Simkin, and Benedikt Wagner. *FRIDA: Data Availability Sampling from FRI*. Cryptology ePrint Archive, Paper 2024/248. https://eprint.iacr.org/2024/248. 2024.

[Sca26] Sofia Scalzo. *A FRI-Based Polynomial Commitment Scheme over Baby-Bear: Reference Implementation*. https://github.com/sofiascalzo/tesi-stark-verifier. 2026.

[Sta21] StarkWare. *ethSTARK Documentation*. Cryptology ePrint Archive, Report 2021/582. https://eprint.iacr.org/2021/582. 2021.

[Tha22] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.html. Foundations, Trends in Privacy, and Security, 2022.